Hashing meta::info

Document #:	P3816R0 [Latest] [Status]
Date:	2025-09-01
Project:	Programming Language C++
Audience:	SG7 Reflection

Reply-to:	Matt Cummins <mcummins16@bloomberg.net> Valentyn Yukhymenko <vyuhimenko@bloomberg.net>

Contents

§ 1 Abstract

This paper proposes a new standard library template, `consteval_hash<T>`, with a single specialization for `meta::info`. The purpose of this facility is to provide a standard interface for compile-time hashing, thereby allowing unordered containers such as `unordered_map` and `unordered_set` to be used with `meta::info` keys, and potentially with other types in future.

```
consteval auto compile_time_function() -> void
{
    const auto hasher = consteval_hash<meta::info>{}; // proposed
    const size_t h = hasher(^^::);

    // now possible
    unordered_map<meta::info, int, consteval_hash<meta::info>> m;
    unordered_set<meta::info, consteval_hash<meta::info>>      s;
}
```

§ 2 Motivation

[P2996] introduces `meta::info` to represent reflections of C++ constructs. However, hashing support was intentionally omitted, as it is not a core reflection feature.

[P3372] makes unordered containers `constexpr`, which creates demand for compile-time hashing in order to use `meta::info` and other consteval-only types as keys.

A straightforward approach would be to specialize `hash<meta::info>`, but `hash<T>` in general is not constexpr-friendly due to its runtime requirements. This paper therefore proposes a dedicated facility for compile-time hashing.

§ 3 Examples

The following examples illustrate practical applications of `consteval_hash`.

§ 3.1 Mp11 `mp_unique` using reflection

In [P2830] (7.3) the authors discuss the infeasibility of implementing `mp_unique` using value-based reflection. Our proposal provides a short and effective solution without needing to sort a list of reflected types.

```
template <typename... Types>
struct type_list {};
```

```
template <typename TypeList>
consteval auto mp_unique_reflected()
{
    static_assert(meta::has_template_arguments(^^TypeList), "mp_unique re-
        quires a type_list");
    static_assert(meta::template_of(^^TypeList) == ^^type_list, "mp_unique
        requires a type_list");

    unordered_set<meta::info, consteval_hash<meta::info>> seen;
    vector<meta::info> unique_types;

    for (auto type_info : meta::template_arguments_of(^^TypeList)) {
          if (const bool is_unique = seen.insert(type_info).second; is_u-
        nique) {
            unique_types.push_back(type_info);
        }
    }

    return meta::substitute(^^type_list, unique_types);
}

template <class TypeList>
using mp_unique = [:mp_unique_reflected<TypeList>():];

using input = type_list<int, char, int, string, double, char>;
using filtered = mp_unique<input>;
using expected = type_list<int, char, string, double>;

static_assert(is_same_v<expected, filtered>);
```

§ 3.2 Custom `name_of` function

In [P2996] (4.4.6), the authors discuss the challenges of producing user-friendly names for reflected entities and argue that functions such as `name_of` should be implemented by third-party C++ libraries rather than standardized, with the standard providing the necessary lower level tools to do so. To implement such a function, one might define a custom mapping of known types or entities to more descriptive labels.

Using `unordered_map` for the mapping provides a terser implementation:

Without map

```
consteval name_of(meta::info r) -
        > string_view
{
    if (r == ^^int) {
        return "integer";
    }
    if (r == ^^float) {
        return "32-bit float";
    }
    if (r == ^^double) {
        return "64-bit float";
```

With map

```
consteval name_of(meta::info r) ->
        string_view
{
    unordered_map<meta::info,
        string_view,
        consteval_hash<meta::info>>
        names = {
        {^^int,      "integer"},
        {^^float,    "32-bit
        float"},
```

Without map

```
    }
    if (r == ^^bool) {
        return "boolean";
    }
    if (r == ^^unsigned) {
        return "unsigned
        integer";
    }
    if (r == ^^::) {
        return "global
        namespace";
    }
    if (r == ^^MyType) {
        return "my library type";
    }
    // add more as required

    // Fall back to identifier if
        it exists
    if (meta::has_identifier(r))
        {
        return
        meta::identifier_of(r);
    }
    return "<unnamed>";
}
```

With map

```
        {^^double,   "64-bit
        float"},
        {^^bool,     "boolean"},
        {^^unsigned, "unsigned
        integer"},
        {^^::,       "global
        namespace"},
        {^^MyType,   "my library
        type"},
        // add more as required
    };

    if (auto it = names.find(r);
        it != names.end()) {
        return it->second;
    }

    // Fall back to identifier if
        it exists
    if (meta::has_identifier(r)) {
        return
        meta::identifier_of(r);
    }
    return "<unnamed>";
}
```

§ 4 Impact

§ 4.1 On the Standard

This proposal is a pure library addition and does not depend on any other library extensions. It introduces the first example of a compile-time hash facility. Moreover, it enables future standard library features: new `consteval` functions may naturally require unordered containers as part of their interfaces, and `consteval_hash` provides the uniform mechanism needed to support such APIs.

§ 4.2 On undefined behavior

This proposal applies only to compile-time programming, where undefined behavior is disallowed. Accordingly, it does not introduce additional undefined behavior into the standard.

§ 4.3 On existing code

As this is a new type in the `std` namespace, this change does not break any existing code, except in the unlikely event that a user has added their own `consteval_hash` type to `std`, which is already undefined behavior.

§ 5 Design decisions

§ 5.1 The API

§ 5.1.1 `hash<meta::info>`

Ultimately, the goal of this paper is to provide a robust way to hash values of type `meta::info`. The most "obvious" way to do this is to implement `hash<meta::info>`, however this introduces inconsistencies:

— Because of the runtime requirements on `hash<T>`, existing specializations cannot be made `constexpr`, meaning we would end up with some specializations of `std::hash<T>` being consteval-only, while others are runtime-only.

— This then makes using the unordered containers at compile-time inconsistent; depending on the key type, you would sometimes be able to use the default hash, while with others you would need to use another.

— When looking at some code involving the unordered containers, readers would not be able to tell just from the code whether the hash is runtime-only or compile-time-only, and would have to look back at the definition of hash. Having a distinct type so that hash is always runtime-only solves this.

§ 5.1.2 `hash_of`

We could sidestep the issues associated with `hash<meta::info>` by instead providing a free function:

```
namespace std::meta {
    auto hash_of(info r) -> size_t;
}
```

This would be defined in `<meta>`. If users need to use `meta::info` as keys in compile-time hash maps, they can use it to implement their own hash (like they will have to do with all other types currently). However, we do not propose this because:

— The functions in `<meta>` provide fundamental details about reflections, and having a `hash_of` function suggests there is a single meaningful way of hashing which is obviously not true. `hash<T>` solves this by simply being understood to provide *a* reasonable default implementation, not *the* implementation.

  — The caveat here is that this is not quite true; `<meta>` also provides `display_string_of`, which provides some reasonable string representation of the given reflection. But aside from this single function, the point still holds.

— More crucially, if we provide users with functionality that they need to wrap themselves, we should just standardize the boilerplate instead. Which leads to…

§ 5.1.3 `consteval_hash<meta::info>`

This brings us to the proposal in this paper. It has a few benefits:

— It is similar to `hash_of`, but does not have the same drawback of suggesting that it is the single meaningful way to hash `meta::info`. It carries the same semantics of a "reasonable implementation" that `hash<T>` has.

— Users won't need to write their own wrapper of `hash_of`.

— Even if `hash<T>` could be made constexpr for other types (which it can't), there would still be an issue with value consistency between runtime and compile-time for pointers, and possibly other types. Separating runtime and compile-time hashing into two separate types avoids this issue, as you shouldn't expect two hashing algorithms to give the same result.

  — Because of this, with a new type you could easily provide `consteval_hash<T*>` as well.

— It can be easily extended for all other standard and builtin types that have a specialization of hash, making compile-time map usage far more uniform.

It also has a few obvious downsides:

— Whenever a new specialization of `hash<T>` is standardized, it likely will also want a corresponding `consteval_hash<T>`, increasing the burden on implementers.

— When users implement `hash<T>` for their own type, they may not care about hash salting and just implement their `hash<T>::operator()` as `constexpr`, which is more natural than also implementing `consteval_hash` and easier to use. However, this is unlikely to be a significant concern for the standard.

— The use of unordered containers in `constexpr` functions remains awkward. Different types are required at compile-time versus runtime, often necessitating heavy use of `if consteval`. However, this is already a problem, and this proposal is not intended to address it.

Overall, this feels like the more complete and extensible solution, so it is the one we are proposing.

§ 5.1.4 Alternate names for `consteval_hash`

There are a few other names we considered. Below is a list of them as well as the reasons we decided against them.

Name	Comments
`stable_hash<T>`	This name would be better suited to a `constexpr` hash usable at both runtime and compile-time. To us, this name does not capture the core feature of the proposed hash, that is to be compile-time only. Our specification for the new type also does not guarantee stability across translation units, so this name is misleading.

`static_hash<T>`	The keyword static in C++ already means "compile-time" in some cases, e.g. `static_assert`, however the keyword is overloaded with many other meanings, so could be confusing. consteval is the keyword for compile-time, hence `consteval_hash`.
`meta_hash<T>`	Concise and clearly related to compile-time functionality. However, the word "meta" more closely relates to reflection, which is a subset of compile-time functionality, and one which compile-time hashing does not necessarily have anything to do with.
`fixed_hash<T>`	Similar to `stable_hash<T>` in meaning.
`compile_time_hash<T>`	This name best describes what it does, but given that C++ already has the `consteval` keyword to mean "compile-time-only", this name is less consistent with the rest of the language.
`comptime_hash<T>`	Although we love Zig, we are proposing a C++ feature!
`ct_hash<T>`	Terse, but too terse. Would you guess that it was a shortening of "compile-time hash" at first glance?
`ce_hash<T>`	Same as above. Would you guess it was a shortening of "consteval hash"?
`compile_time_only_hash<T>`	Consistent with `move_only_function`, but far too long.
`consteval_only_hash<T>`	A slight improvement on the above, but the "only" is superfluous since consteval already denotes that it only works at compile-time.
`constexpr_hash<T>`	Just incorrect, implies that it is usable at runtime too. If such a type existed, you would expect its interface to be made up of `constexpr` functions, not `consteval`.
`hash<T, hashtype::compile_time>`	Rather than a new type, we could instead extend `hash<T>` by providing an enum class to select what kind of hash you want. This enum could be extended to provide even more hashes in the future. This feels far messier, and given that it would be impossible to implement certain hashes for certain types, it would be misleading and provide an API that looks incomplete.

§ 5.2 Ordered maps and sets

Given that this proposal focuses on enabling unordered maps and sets keyed on `meta::info`, it is natural to also ask what meaning, if any, should be assigned to `map<meta::info, T>` and `set<meta::info>`. Both of these would require `less<meta::info>` to be well-defined, which would require `operator<` to be defined. There is no meaningful natural ordering for reflections, but we discussed a couple of options here:

— A naive approach would be to implement `operator<` in terms of `consteval_hash<meta::info>`, but the issue is that distinct reflections are not guaranteed to have distinct hashes. The ordering would change every time the hash does as well.

— A better approach would be to define `operator<` in the same way that it is defined for pointers, which is done via an implementation defined strict total ordering, with no guarantees given on the consistency of ordering between runs.

Regardless of how `operator<` is implemented, an explicit specialization for `less<meta::info>` would still be required in order to make it a consteval-only type.

In addition to that, [P2830] (4.1.4) states that any `operator<=>` defined for `std::meta::info` should be consistent with compile-time type ordering it proposes.

Ultimately this seems like a harder problem and we intentionally leave this out of scope, restricting this paper to hashing.

§ 6 Proposed wording

§ 6.1 The specification for `consteval_hash<T>`

Add a new section, `ConstevalHash` [constevalhash.requirements], defined analogously to `Cpp17Hash` [hash.requirements]:

> A type `H` meets the `ConstevalHash` requirements if
>
> 1 — It is a function object type ([function.objects]).
>
> 2 — It meets the `Cpp17CopyConstructible` and `Cpp17Destructible` requirements.
>
> 3 — It is a consteval-only type ([basic.types.general]).
>
> 4 — Given two instances of H, it is not guaranteed that they will produce the same values for the same arguments.
>
> (4.1) — [*Note:* In particular, the values may be different between translation units. — *end note*]
>
> 5 — The expressions in the table below are valid and have the indicated semantics:
>
> Given `Key` is an argument type for function objects of type `H`, `h` is a value of type (possibly `const`) `H`, `u` is an lvalue of type `Key`, and `k` is a value of type

convertible to (possibly `const`) `Key`.

Expression	Return type	Requirement
`h(k)`	`size_t`	The value returned shall depend only on `k`. The value shall be stable across repeated compiler runs. [*Note:* Modifying the source code may change the value. — *end note*] For two different values `t1` and `t2`, the probability that `h(t1)` and `h(t2)` compare equal should be very small, approaching `1.0 / numeric_limits<size_t>::max()`.
`h(u)`	`size_t`	Shall not modify `u`.

Add a new section, "Class template consteval_hash" [unord.consteval_hash], defined analogously to "Class template hash" [unord.hash]. The only difference is that this currently makes no mention of specializations for `nullptr_t` or for cv-unqualified arithmetic, enumeration and pointer types (which can be added later):

Class `template consteval_hash`

1 — Each specialization of `consteval_hash` is either enabled or disabled, as described below.

(1.1) — [*Note:* Enabled specializations meet the `ConstevalHash` requirements, and disabled specializations do not. — *end note*]

2 — If the library provides an explicit or partial specialization of `consteval_hash<Key>`, that specialization is enabled except as noted otherwise, and its member functions are `noexcept` except as noted otherwise.

3 — If `H` is a disabled specialization of `consteval_hash`, these values are `false`: `is_default_constructible_v<H>`, `is_copy_constructible_v<H>`, `is_copy_assignable_v<H>`, and `is_move_assignable_v<H>`. Disabled specializations of `consteval_hash`> {.cpp} are not function object types ([function.objects]).

(3.1) — [*Note:* This means that the specialization of consteval_hash exists, but any attempts to use it as a ConstevalHash will be ill-formed. — *end note*]

4 — An enabled specialization `consteval_hash<Key>` will:

(4.1) — Meet the `ConstevalHash` requirements, with `Key` as the function call argument type, the `Cpp17DefaultConstructible` requirements, the `Cpp17CopyAssignable` requirements, the `Cpp17Swappable` requirements.

(4.2) — Meet the requirement that if `k1 == k2` is `true`, `h(k1) == h(k2)` is also `true`, where `h` is an object of type `consteval_hash<Key>` and `k1` and `k2` are objects of type Key.

(4.3) — Meet the requirement that the expression `h(k)`, where `h` is an object of type `consteval_hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `consteval_hash<Key>` is a program-defined specialization.

§ 6.2 The specialization for `meta::info`

Add a new section [meta.reflection.hash]:

```
template <typename T> struct consteval_hash;
template <> struct consteval_hash<meta::info>;
```

The specialization is enabled ([unord.consteval_hash]).

§ 6.3 Feature testing macros

Add two new feature macros into [version.syn], one for the new type, and one for the `meta::info` instantiation:

```
#define __lib_consteval_hash_template YYYYXXL  // also in <meta>
#define __lib_consteval_hash_meta_info YYYYXXL // also in <meta>
```

§ 7 Implementation experience

We implemented this on a branch of Bloomberg's Clang fork. Like with the rest of the reflection API, `consteval_hash<meta::info>::operator()` can be implemented via a compiler intrinsic.

Our initial implementation was unstable across repeated runs due to it hashing pointers under the hood, but was simple. To implement a hash that was stable across repeated compiler runs, we assigned each `Type` a unique ID which made hashing type reflections stable. For most other reflections, we were able to define the hash based on their source locations which were also stable. Naturally, the values change if code is moved or new type definitions are added, but this still satisfies the requirement of not breaking repeat builds.

```
template<>
struct consteval_hash<meta::info>
{
  consteval consteval_hash() = default;
  consteval consteval_hash(const consteval_hash<meta::info>&) = default;
  consteval consteval_hash(consteval_hash<meta::info>&&) = default;

  consteval auto operator()(meta::info r) const noexcept -> size_t {
    return __metafunction(meta::detail::__metafn_reflection_hash, r);
  }
```

```
private:

  // This unused variable is here to make consteval_hash<> a
  // consteval-only type.
  [[maybe_unused]] const meta::info unused = ^^::;
};
```

A simpler alternative would be to hash the display strings associated with each reflection, for example using `meta::display_string_of` or a similar function. However there is no guarantee that this function provides good quality names that can provide a quality hash, but this would produce values that are far more stable, including across translation units.

[P3068] has been approved for C++26, allowing exception throwing within constexpr, so it is meaningful to mark `consteval_hash<meta::info>::operator()` as `noexcept`.

§ 8 Acknowledgements

— Dan Katz for the original implementation and wording, as well as his work on implementing the main reflection paper in Clang.

— Hana Dusíková for allowing unordered containers to be usable in constexpr, which is a primary motivator for this paper.

§ 9 References

[P2830] Nate Nichols and Gašper Ažman. Constexpr Type Ordering.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2830r10.html

[P2996] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. Reflection for C++.
https://isocpp.org/files/papers/P2996R13.html

[P3068] Hana Dusíková. Allowing Exception Throwing in Constant-Evaluation.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3068r2.html

[P3372] Hana Dusíková. Constexpr Containers and Adaptors.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3372r3.html